

Bayesian AdaChain Under Unreliable Node Reports

This report evaluates an AdaChain extension for settings where the true workload is not directly observed. Each episode has a latent workload state, multiple node reports, a robust Bayesian filter over the workload, and a learned Random Forest reward model for architecture selection.

The main result is nuanced. Multiple imputation is not consistently strong. A simpler uncertainty-aware variant, trained on posterior mean and posterior standard deviation features, is stronger in reliability-drift and biased-outlier scenarios, but simple averaging still performs well in some noise-only cases.

The correct interpretation is not that this is a classical Byzantine-defense mechanism. The model addresses unreliable, noisy, stale, biased, or drifting node reports that remain statistically related to the true workload.

Updated Algorithm

1. At episode t , the true workload x_t is hidden. Nodes report noisy workload estimates $o_{\{i,t\}}$.
2. A robust Bayesian filter consumes all node reports and estimates a belief over the latent workload. The implemented belief is approximated by posterior mean $E[x_t | o_t]$ and posterior standard deviation $Std[x_t | o_t]$.
3. The uncertainty-aware reward model trains a RandomForestRegressor on feature vectors [posterior_mean, posterior_std, architecture_features] -> observed throughput.
4. At decision time, the filter draws posterior workload samples. For each candidate architecture, the Random Forest predicts throughput for sampled workload contexts while retaining the posterior uncertainty features. Predictions are averaged to approximate expected value under the belief state.
5. The selected architecture is executed in the simulator. The observed throughput is stored with the posterior mean, posterior standard deviation, and selected architecture. The Random Forest is retrained on a sliding window.

Compared with the earlier multiple-imputation approach, this avoids attaching the same reward to many sampled contexts. It keeps training cleaner while still exposing the policy to uncertainty.

Exact Algorithm Mechanics

Agents compared in the report. Oracle sees the true latent workload and is an upper-reference policy for expected reward. Mean reports averages all node reports and feeds that context into the AdaChain-style Random Forest bandit. Median reports uses the coordinate-wise median. Bayes imputation uses the robust posterior, stores multiple sampled latent histories, and trains one Random Forest per imputed history. Bayes mean+std is the new uncertainty-feature model.

Workload/context vector. Every episode has $x = [\text{write_ratio}, \text{hot_key_ratio}, \text{arrival_rate}, \text{execution_delay}]$. Reports are $N \times 4$ matrices, one four-dimensional estimate per node. The architecture/action features are $[\text{blocksize}, \text{early_execution}, \text{reorder}, \text{signed_blocksize}]$, where signed_blocksize is positive when early execution is enabled and negative otherwise.

Robust Bayesian filter implementation. The estimator first subtracts the previous mean-reverted node bias from each report. It initializes the latent context with the coordinate-wise median. It then performs four robust reweighting iterations. For each node and feature, residuals are scored with Student-t style weights: $\text{weight} = (\nu + 1) / (\nu + (\text{residual} / \text{scale})^2)$. Large residuals therefore receive lower precision. The weighted precision average gives the posterior mean approximation.

Reliability state. The filter keeps node-feature bias and noise-scale estimates. Bias follows mean reversion with $\rho_b = 0.95$ and is constrained to sum to zero across nodes per feature, so the latent context remains identifiable as the consensus workload. Noise scale follows a smoothed update with $\rho_{\sigma} = 0.95$. Posterior standard deviation is computed from the inverse summed precision and floored to avoid pretending certainty is exact.

Bayes mean+std training. After action execution, the stored supervised example is $[\text{posterior_mean}, \text{posterior_std}, \text{selected_action_features}] \rightarrow \text{observed_reward}$. This is a posterior-summary feature augmentation method: the reward model sees both the inferred workload and how uncertain that inference was.

Bayes mean+std decision rule. For each episode, the filter draws posterior context samples. For each candidate action, the Random Forest predicts reward for each sampled context while appending the same posterior_std and that action's features. The action score is the average predicted reward over samples. The selected architecture is the argmax score.

Why this differs from multiple imputation. Multiple imputation attaches one observed reward to several sampled contexts, which can blur the reward surface when the posterior is wide. The mean+std version stores one cleaner training point per episode while still allowing uncertainty to affect both training and decision-time scoring.

Exact Data Generation Scenarios

Shared simulator setup. Each scenario starts from the same hidden workload generator and the same report generator structure. There are 9 reporting nodes by default. The unreliable fraction is 0.25, so $\text{ceil}(9 * 0.25) = 3$ nodes are marked unreliable. All reports are clipped to legal workload bounds. Reliable nodes report `true_context` plus small Student-t noise and a small initialized node bias. The base feature noise scale is [0.035, 0.035, 250, 350] for [write_ratio, hot_key_ratio, arrival_rate, execution_delay].

`biased_outliers`. This is the default heavy-bias scenario. Each unreliable node receives a fixed random sign per feature. Its report is `true_context + 1.8 * sign * unreliable_bias_scale + heavy-tailed Student-t noise`. The unreliable bias scale is [0.18, 0.18, 900, 2200]. This creates consistently wrong, heavy-tailed reports, but not strategic adversarial behavior.

`persistent_bias`. At initialization, unreliable nodes receive persistent feature-wise offsets equal to `sign * unreliable_bias_scale`. During sampling they still follow the true context plus noise, but from a shifted reporting baseline. This tests whether a method can handle nodes that are systematically calibrated wrong.

`heterogeneous_noise`. Unreliable nodes do not necessarily have large fixed bias. Instead, their noise multiplier is [3, 3, 4, 4], so their write/hot-key reports are three times noisier and their rate/delay reports are four times noisier. This tests unequal reliability across nodes.

`reliability_drift`. Unreliable nodes change behavior by phase. Every 50 episodes, the unreliable nodes move between phases. In phase 1, their noise multiplier becomes [4, 4, 5, 5] and their reported context is shifted by `0.7 * sign * unreliable_bias_scale`. In phase 2, their noise multiplier becomes [1.5, 1.5, 2, 2]. In phase 0, they reset closer to normal. This is the scenario most aligned with the Bayesian filter's mean-reverting reliability state.

`stale_reports`. Once enough history exists, unreliable nodes report an old true context instead of the current true context. The default lag is 4, so they report the workload from five stored positions back. This simulates delayed/stale monitoring rather than random corruption.

`outlier_bursts`. Unreliable nodes usually behave like normal noisy nodes, but with probability 0.10 per episode they add a burst equal to `sign * unreliable_bias_scale * U(1.0, 2.5)`. This tests rare transient failures.

`mixed_unreliable`. This combines persistent bias, heterogeneous noise, reliability drift, stale reports, and burst outliers. It is a stress scenario for structured but non-adversarial unreliability.

What The Plots And Tables Mean

Total regret is the most important comparison metric. In each episode, the simulator computes the expected throughput of the best action under the true hidden workload. Regret is $\text{best_expected_throughput} - \text{selected_action_expected_throughput}$, clipped at zero. Lower total regret means the policy chose architectures closer to the true best architecture. This avoids being fooled by random reward noise.

Total reward is the sum of observed sampled rewards. It is useful, but noisy. Because reward includes stochastic noise, an agent can sometimes have higher realized reward than the oracle even when its expected decisions are worse. This is why regret is treated as the cleaner performance metric.

Mean context error is the average L2 distance between the agent's estimated workload and the hidden true workload. It measures state-estimation quality, not action quality. The report shows that better context estimates do not always imply better reward, because the downstream Random Forest policy also has to learn the reward surface correctly.

Architecture accuracy is the fraction of episodes where the agent chose the same action as the expected-throughput oracle. This is strict: two architectures can have similar expected reward, but only the exact oracle action counts as correct.

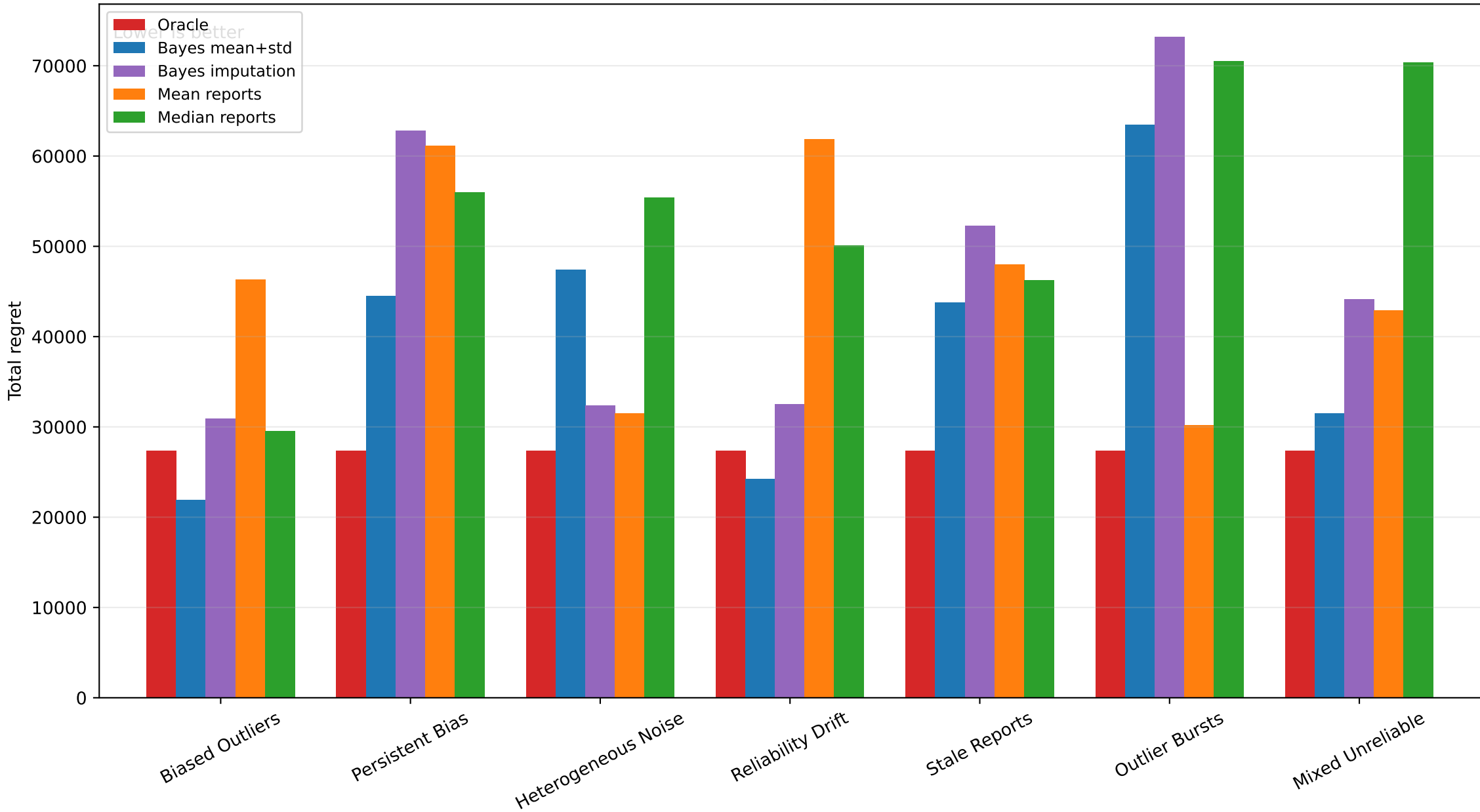
Cumulative reward/regret plots show how performance evolves over time. A flatter cumulative regret curve means the agent is making fewer costly action mistakes. Separation late in training indicates that the learned policy is exploiting accumulated experience differently across report scenarios.

The key story shown by the report is that the uncertainty-feature model is not merely estimating context; it exposes posterior uncertainty to the reward predictor. This helps especially under reliability drift, persistent bias, stale reports, and mixed unreliability. Mean or median baselines remain competitive in simpler noise patterns, which is important for honest evaluation.

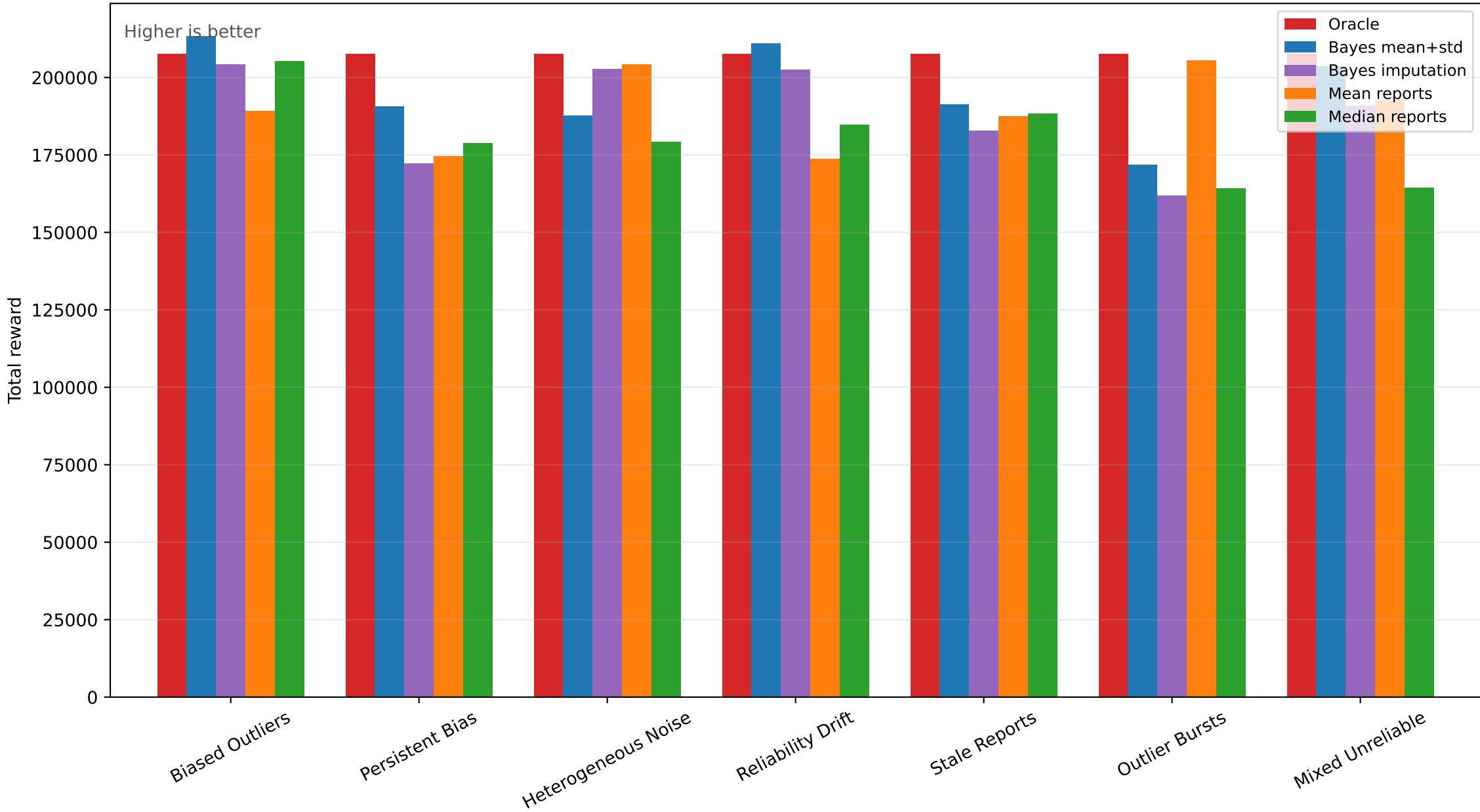
Best Non-Oracle Agent by Regret

scenario	agent	total_regret	total_reward	mean_context_error	architecture_accuracy
Biased Outliers	Bayes mean+std	21,900.6	213,234.4	486.2	0.4
Heterogeneous Noise	Mean reports	31,460.8	204,103.0	380.0	0.0
Mixed Unreliable	Bayes mean+std	31,503.8	203,648.0	383.1	0.2
Outlier Bursts	Mean reports	30,167.2	205,434.5	225.2	0.0
Persistent Bias	Bayes mean+std	44,476.2	190,658.7	313.5	0.2
Reliability Drift	Bayes mean+std	24,243.3	210,891.6	190.1	0.7
Stale Reports	Bayes mean+std	43,801.7	191,333.2	256.7	0.4

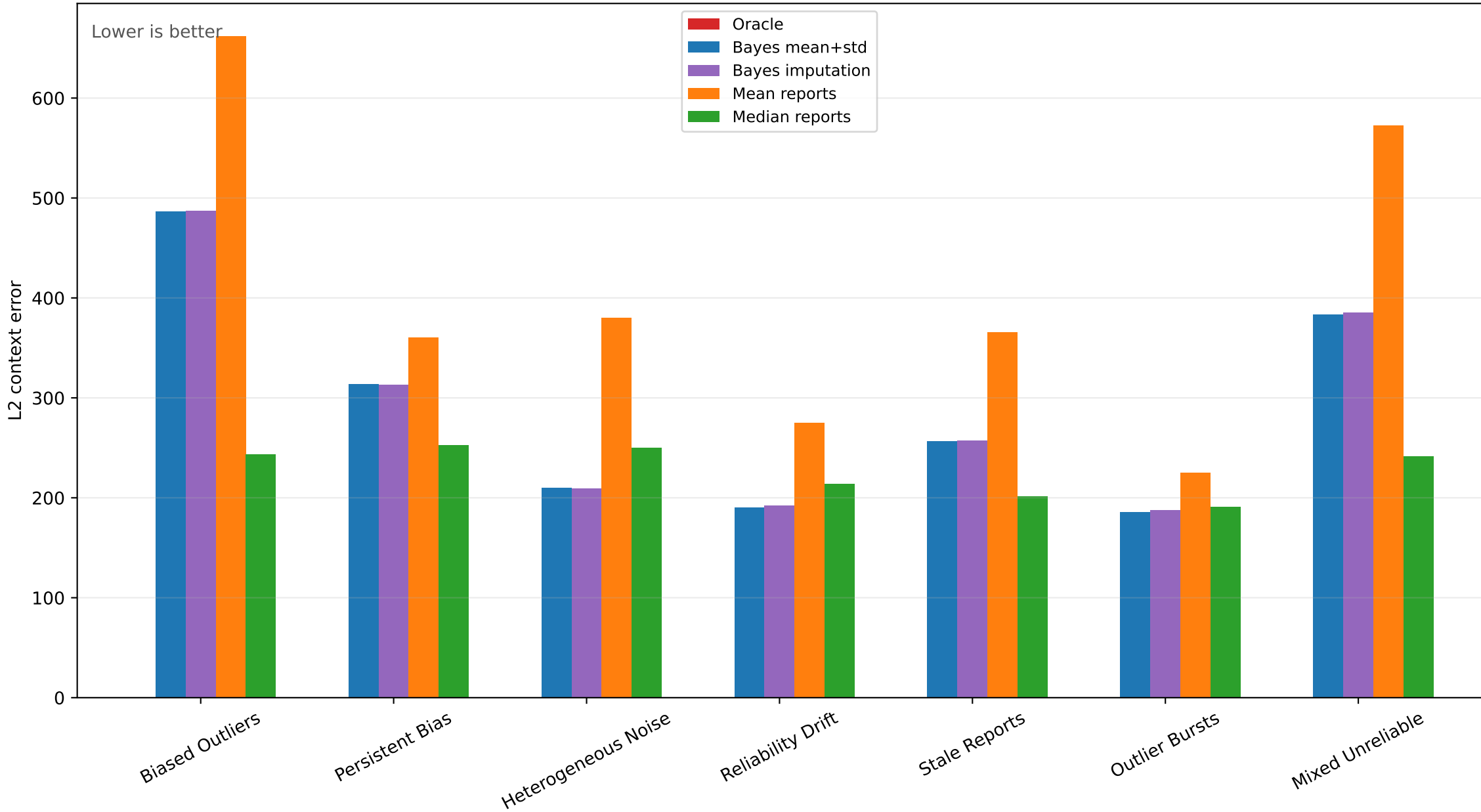
Total Regret by Scenario



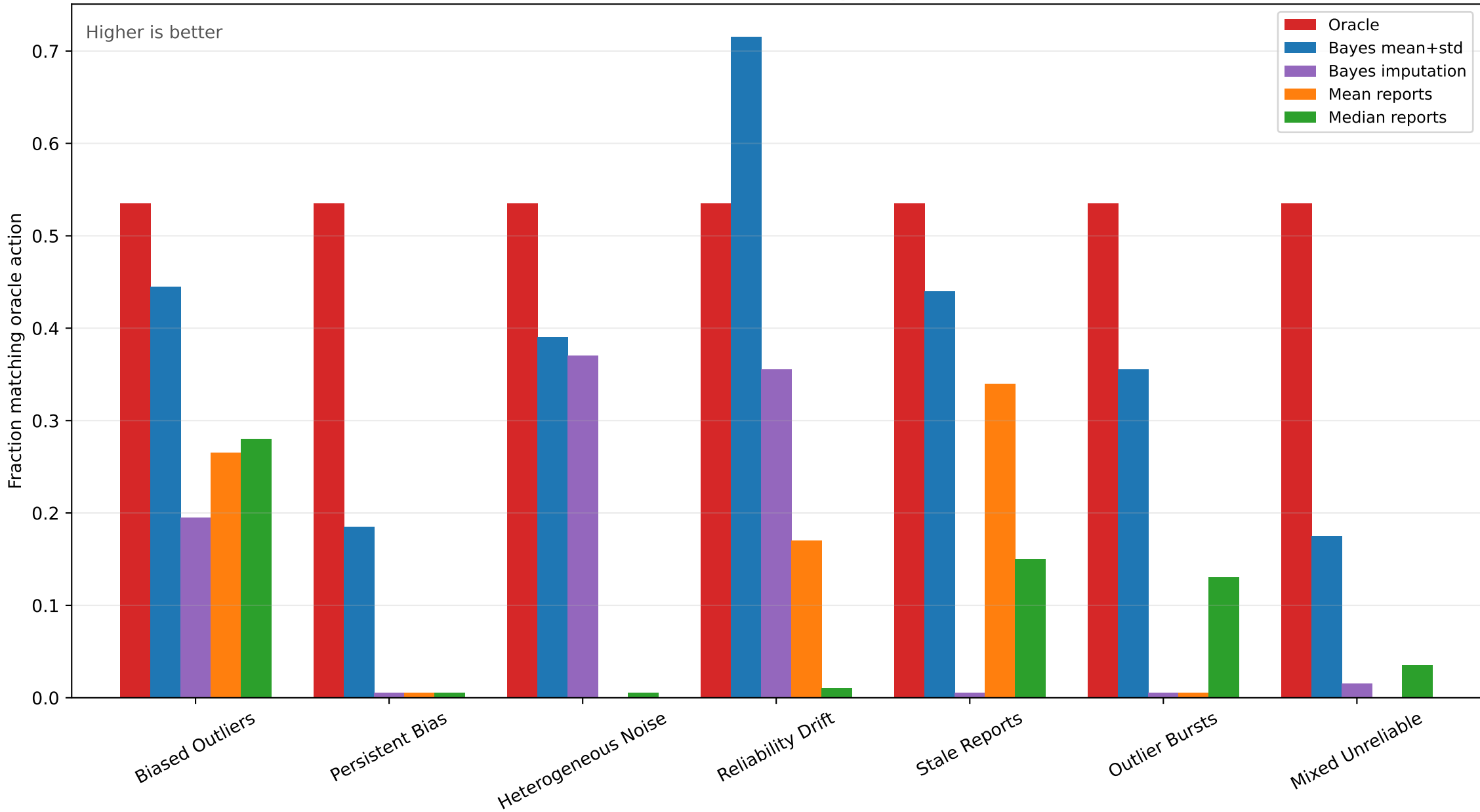
Total Reward by Scenario



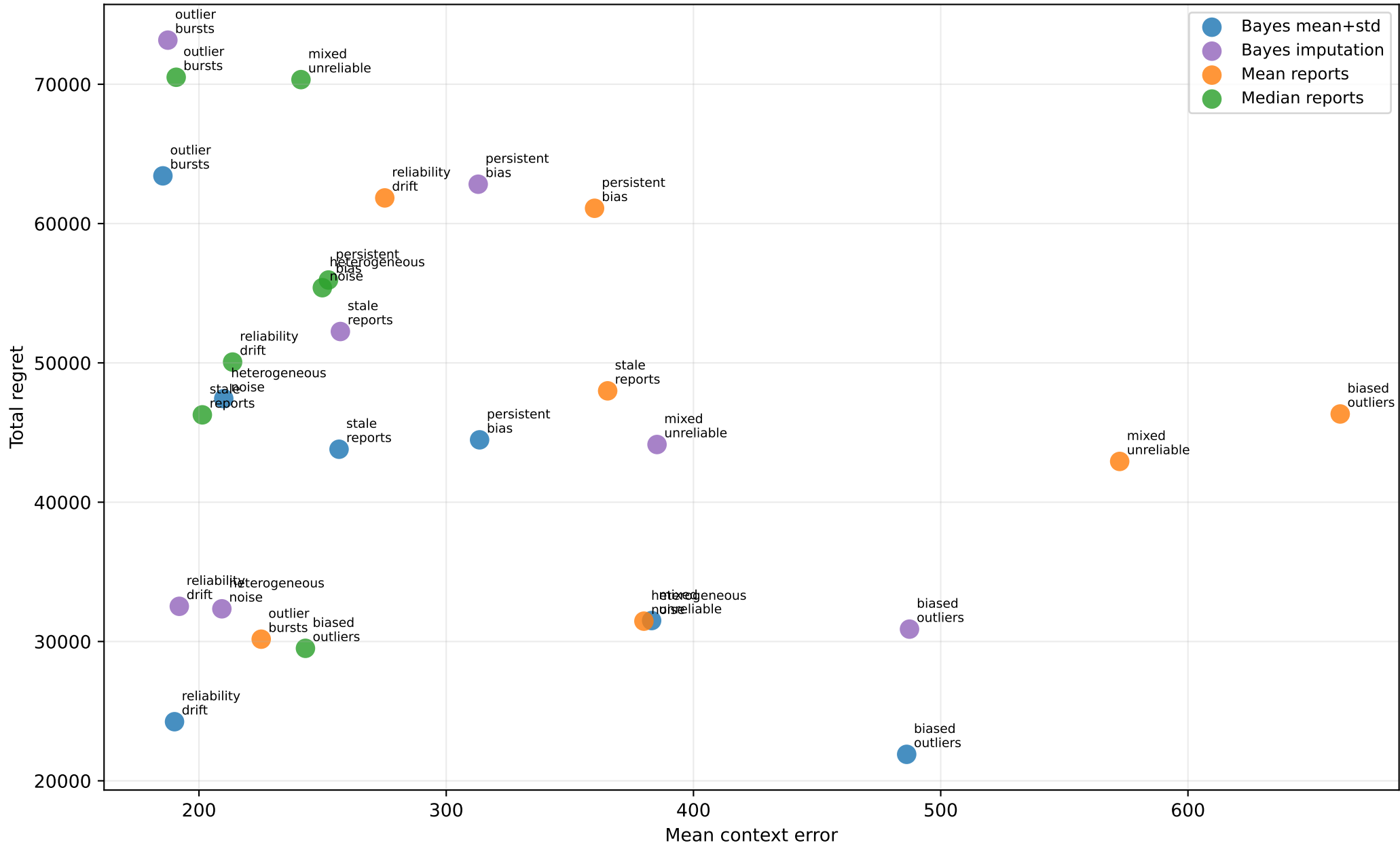
Mean Context Error by Scenario



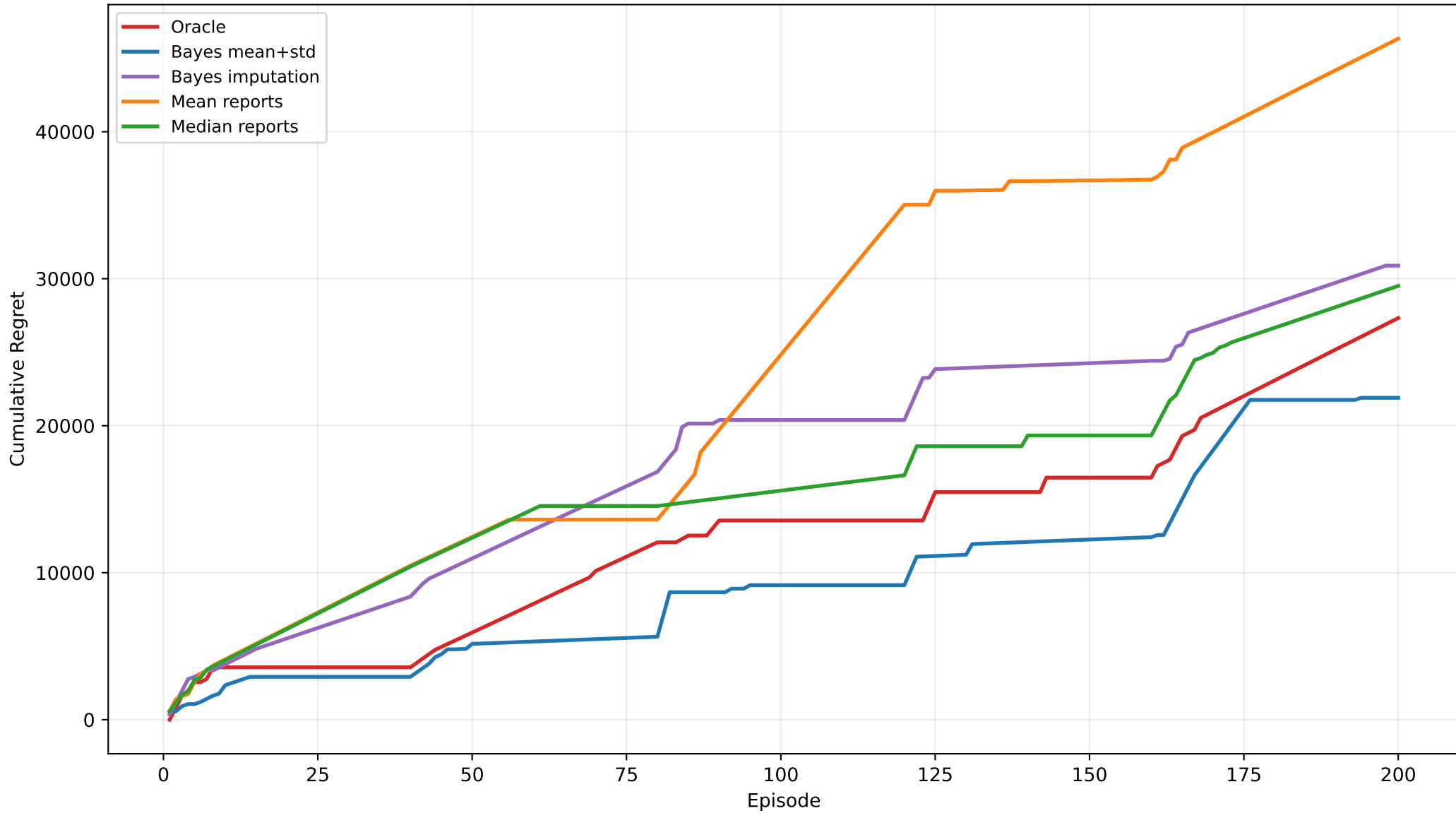
Architecture Accuracy by Scenario



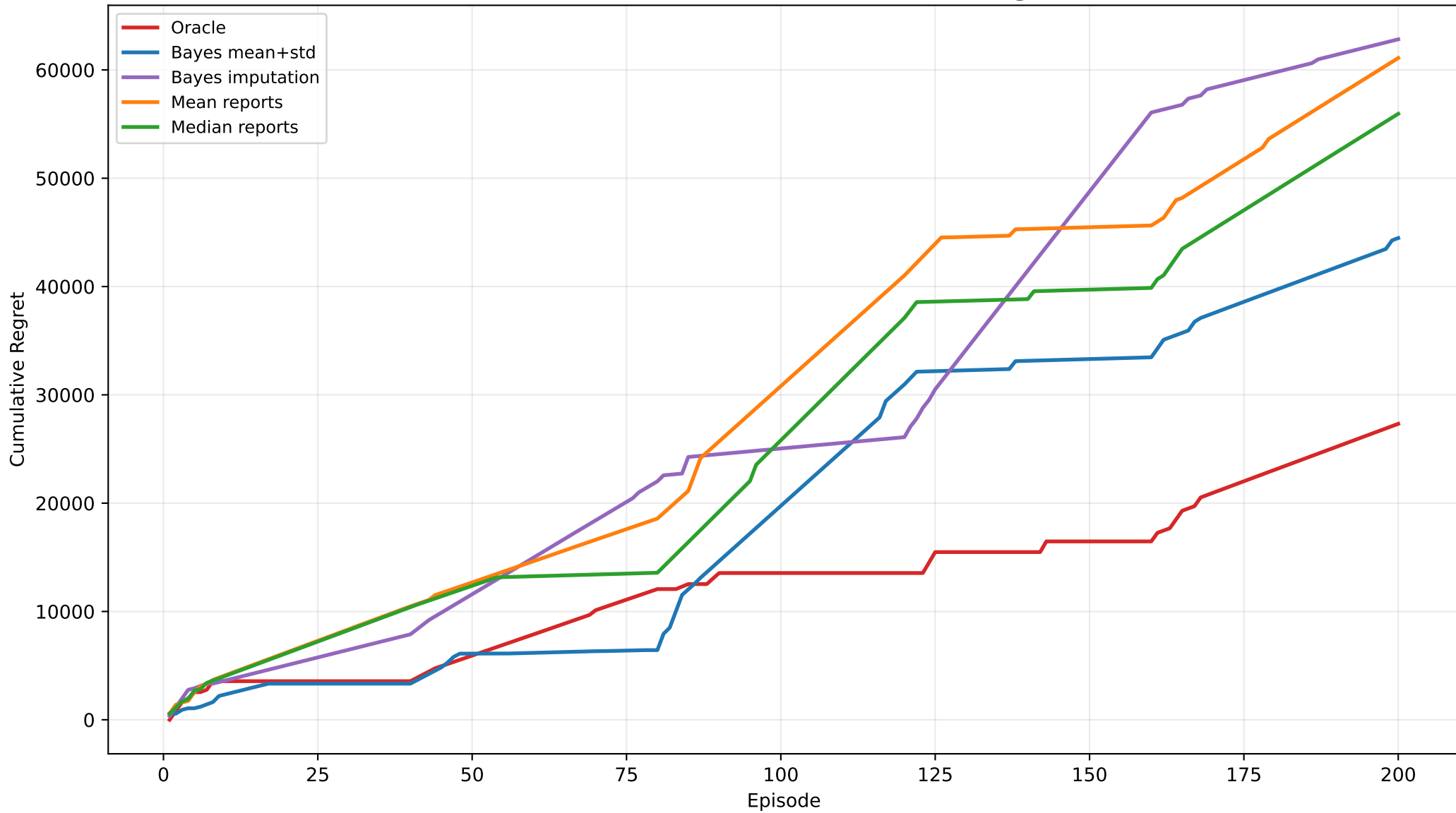
Context Accuracy Does Not Always Translate to Reward



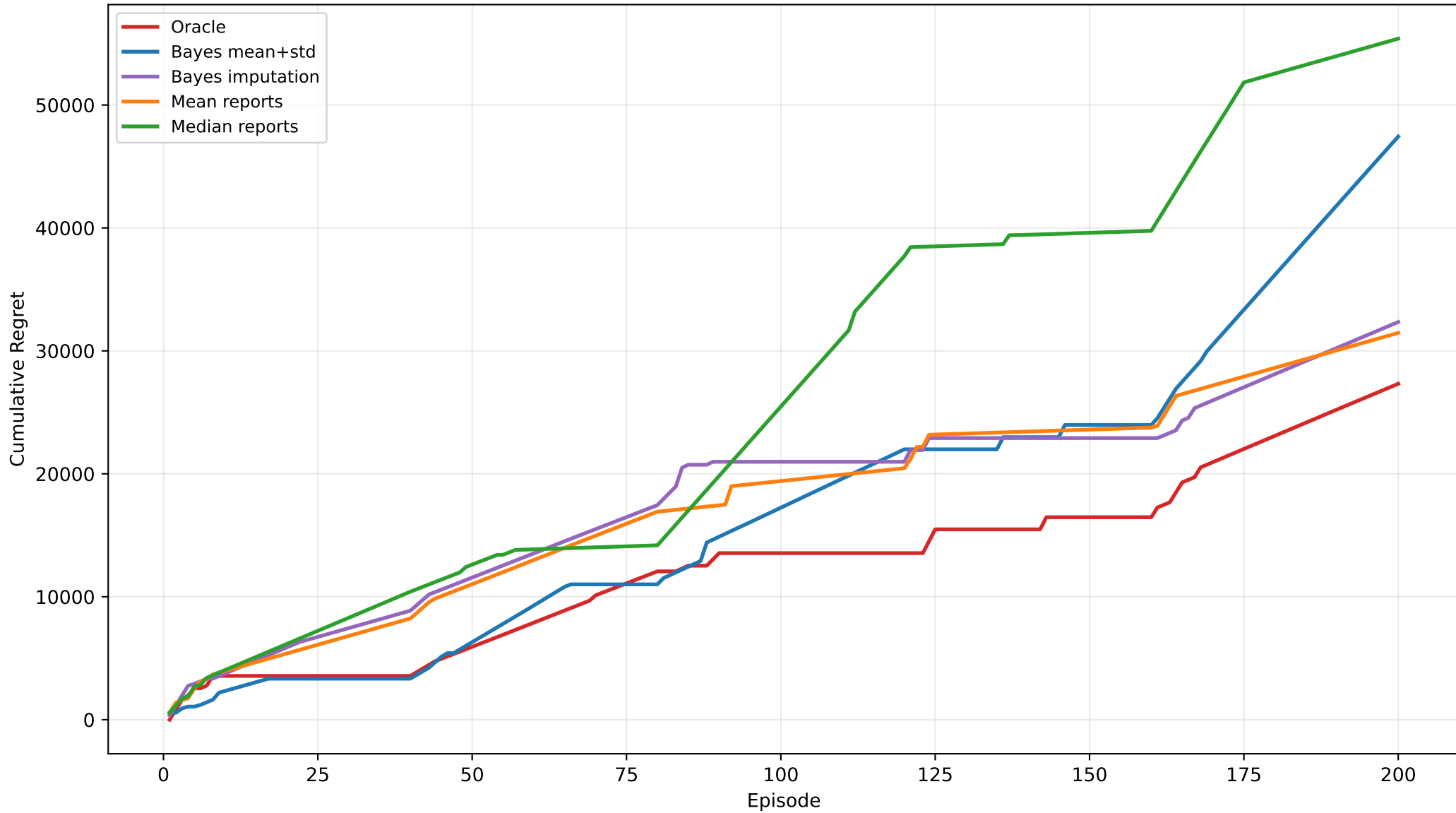
Biased Outliers: Cumulative Regret



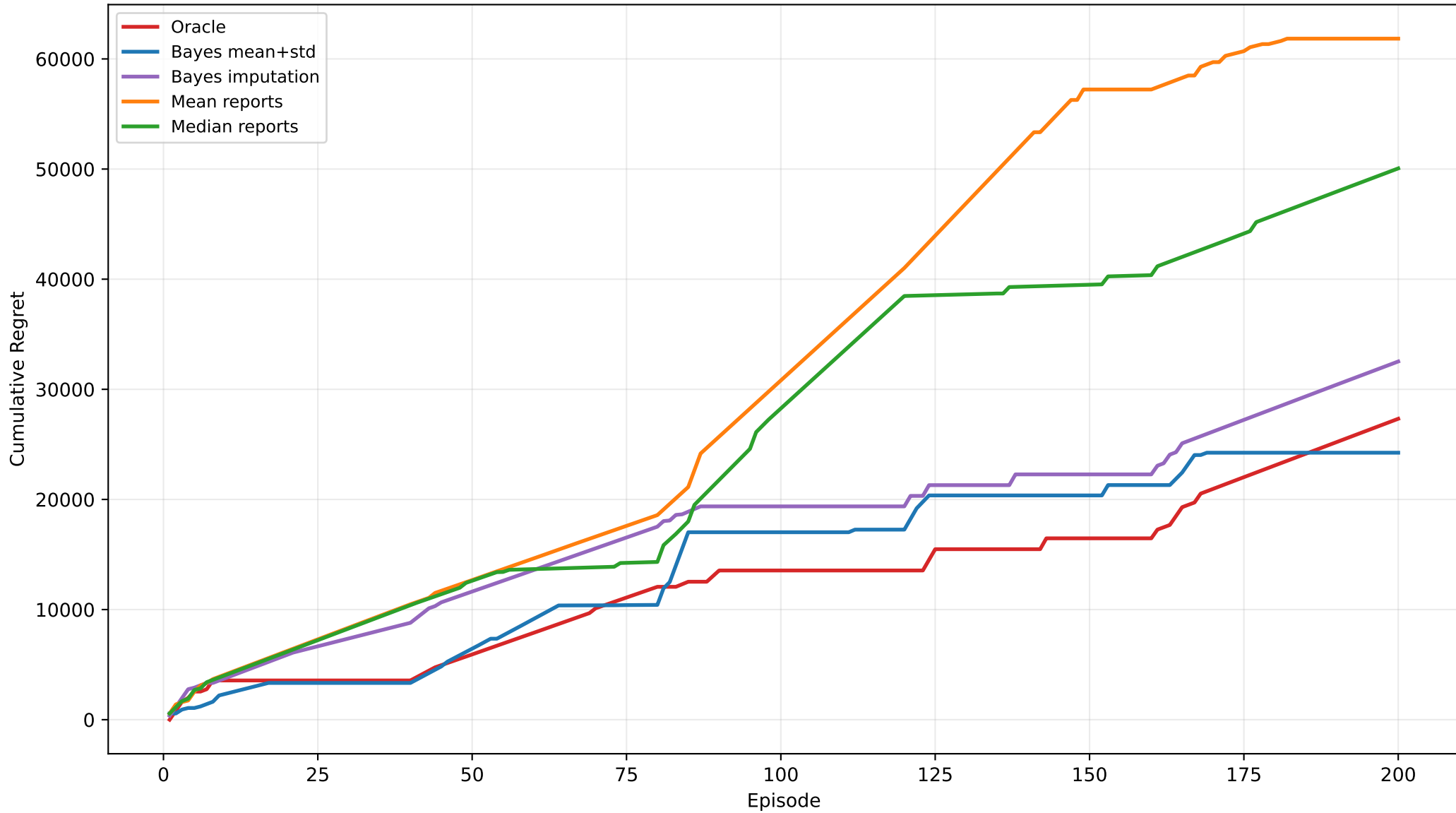
Persistent Bias: Cumulative Regret



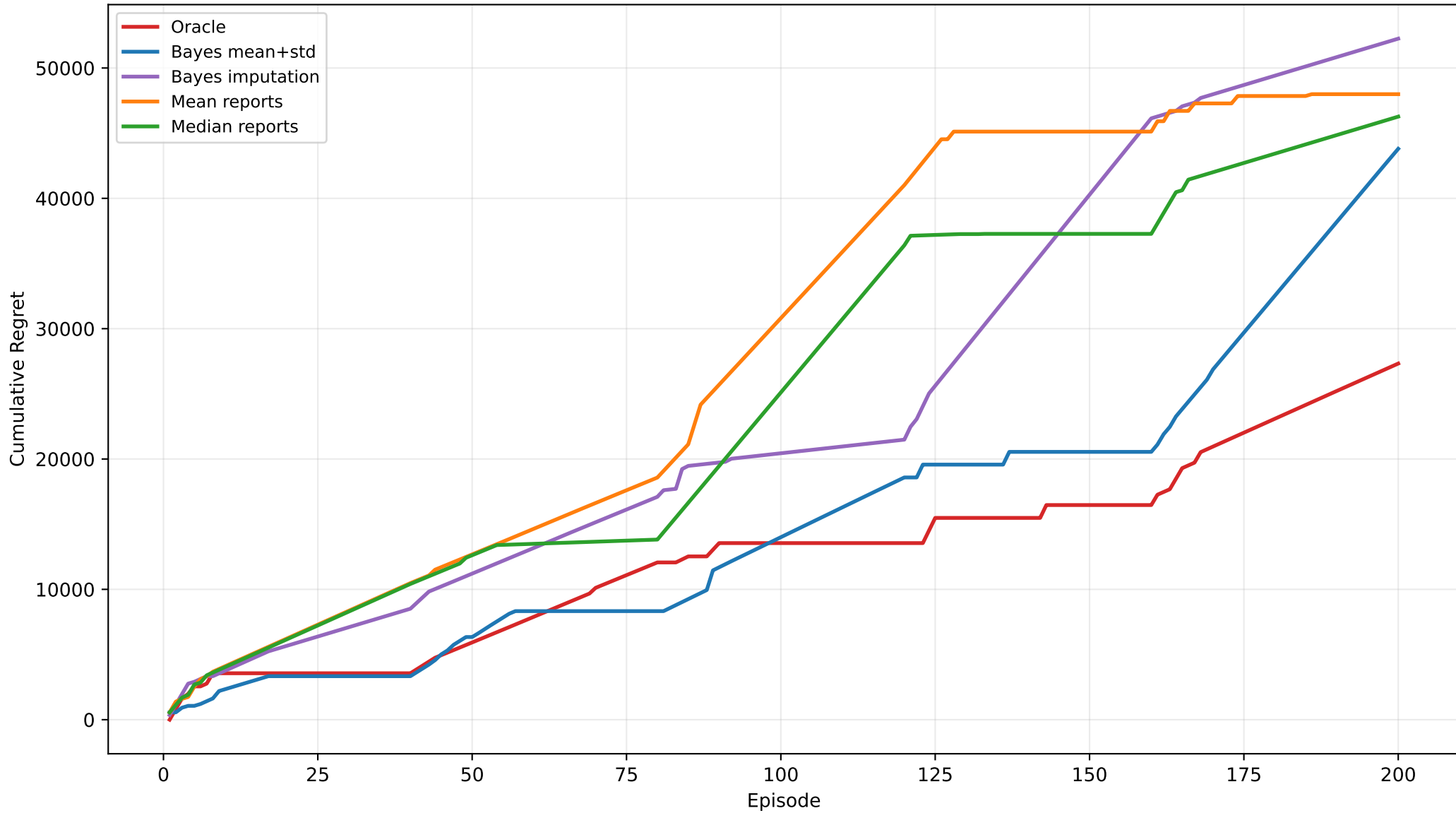
Heterogeneous Noise: Cumulative Regret



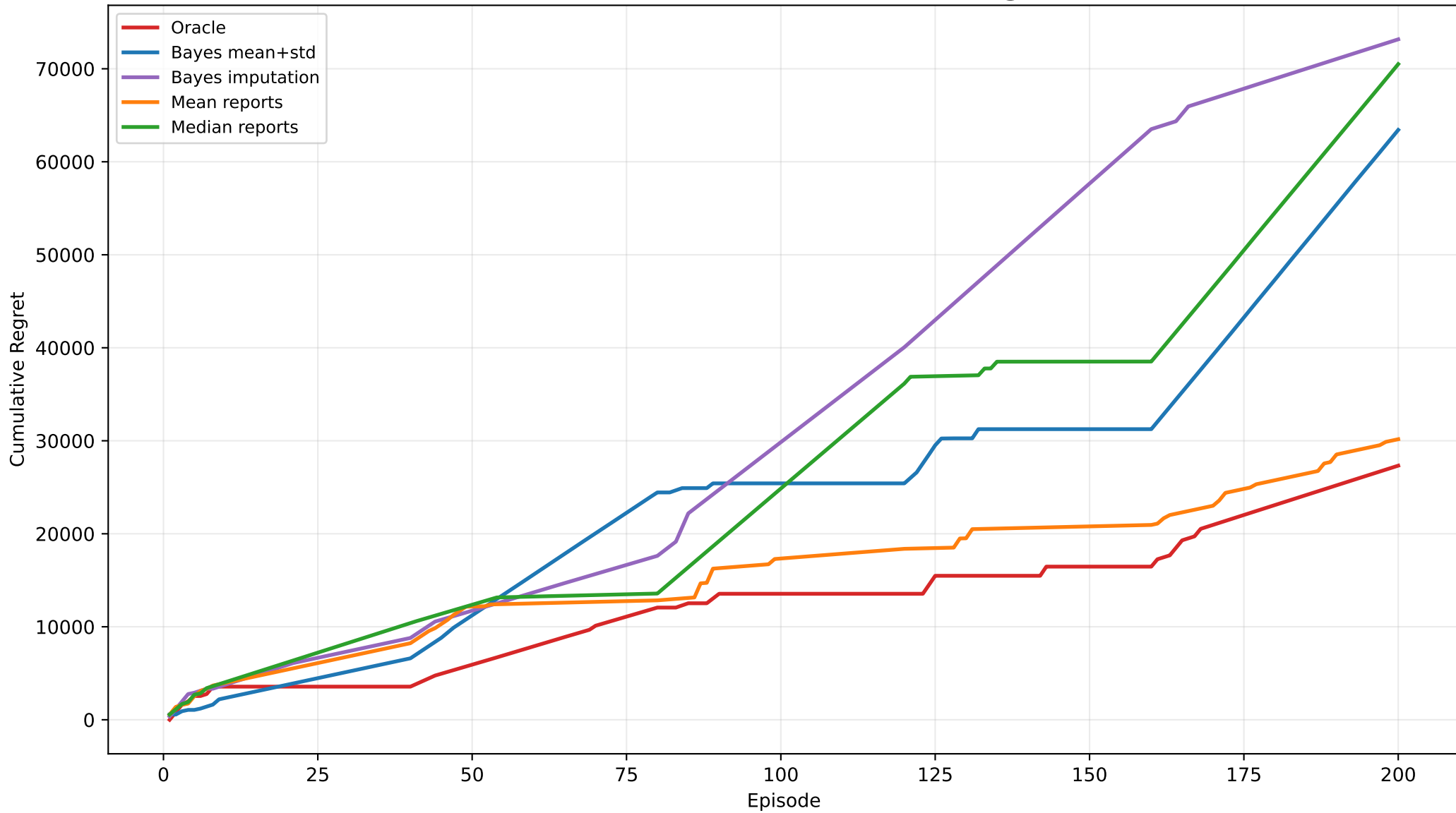
Reliability Drift: Cumulative Regret



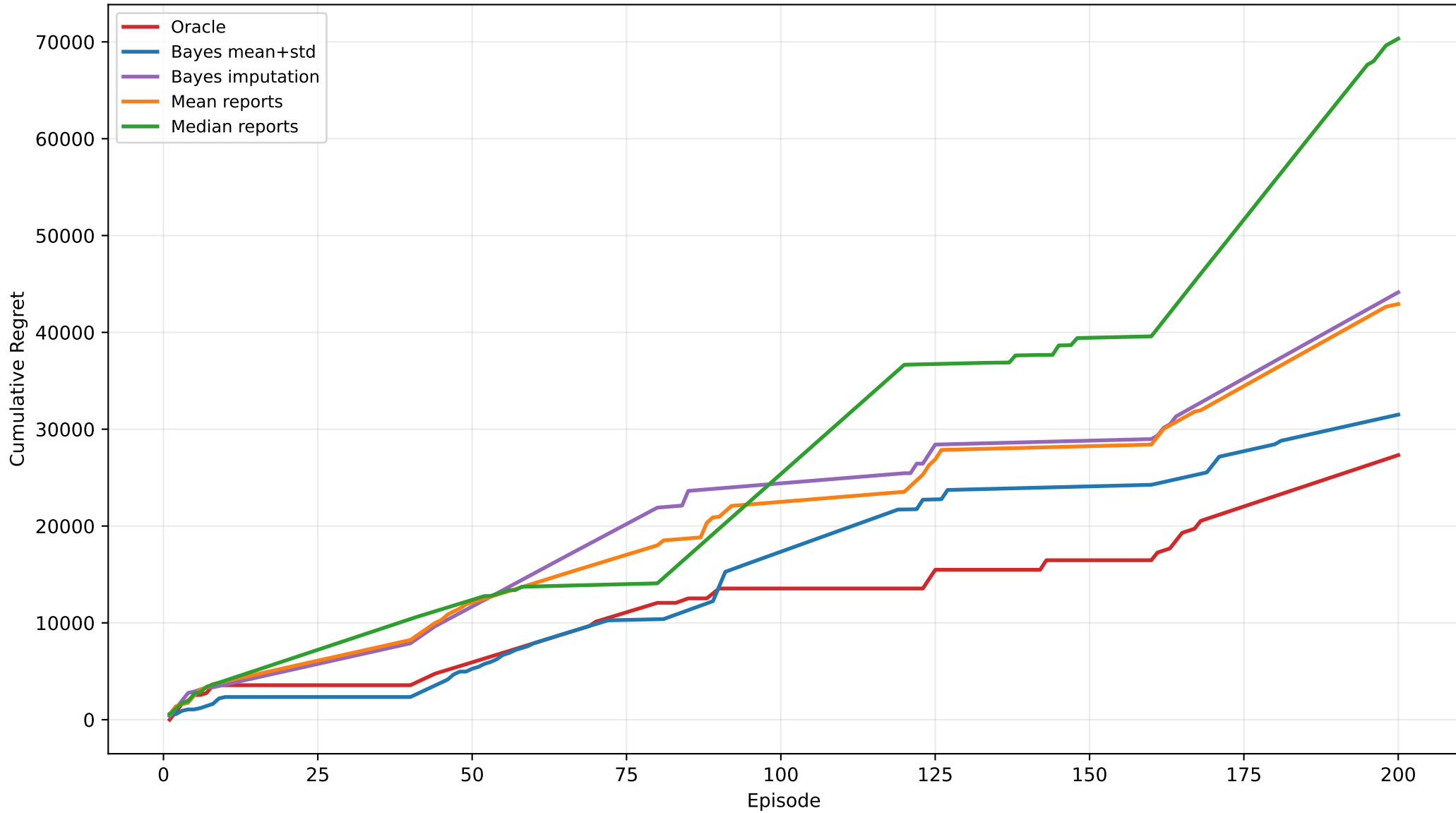
Stale Reports: Cumulative Regret



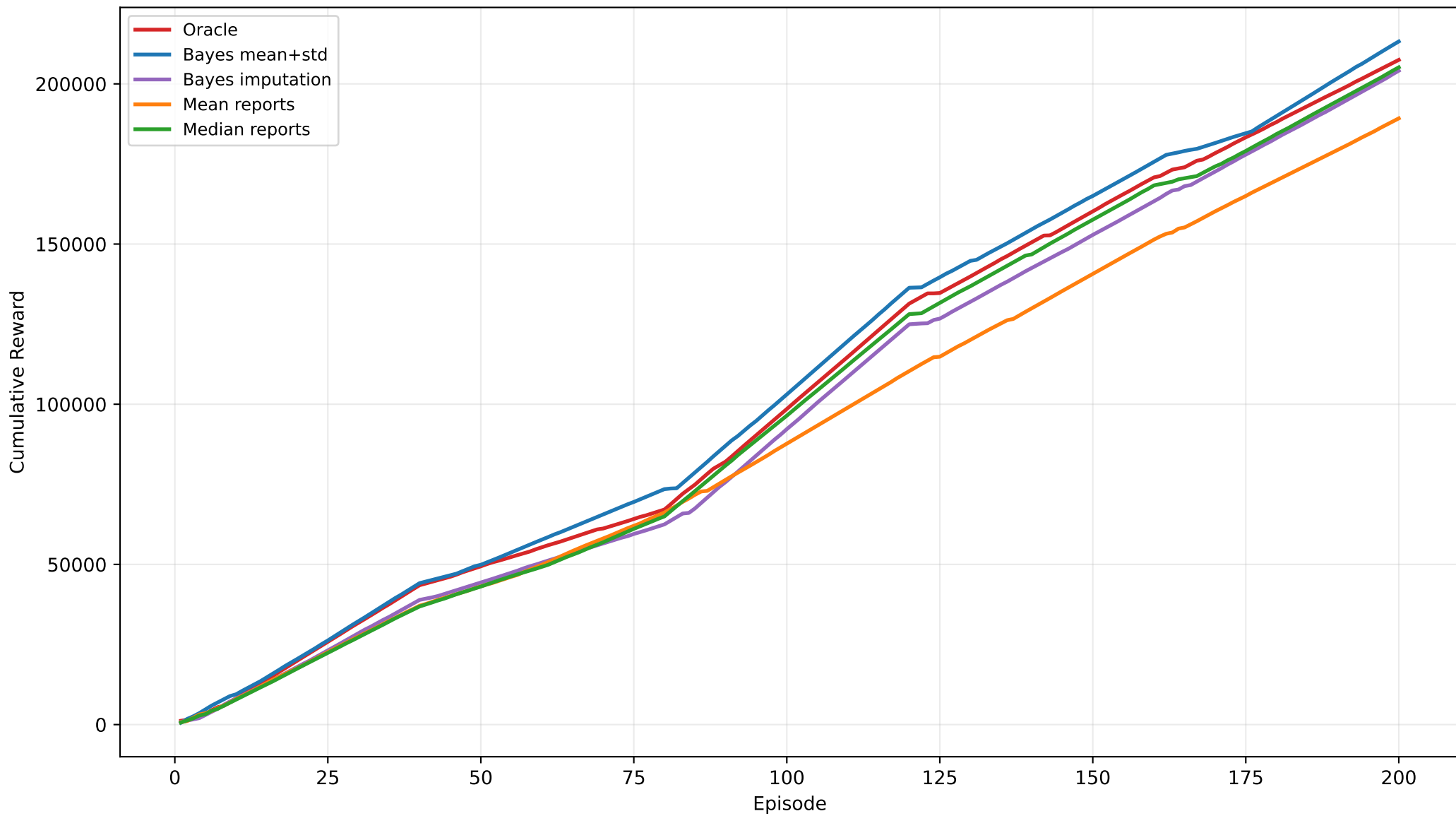
Outlier Bursts: Cumulative Regret



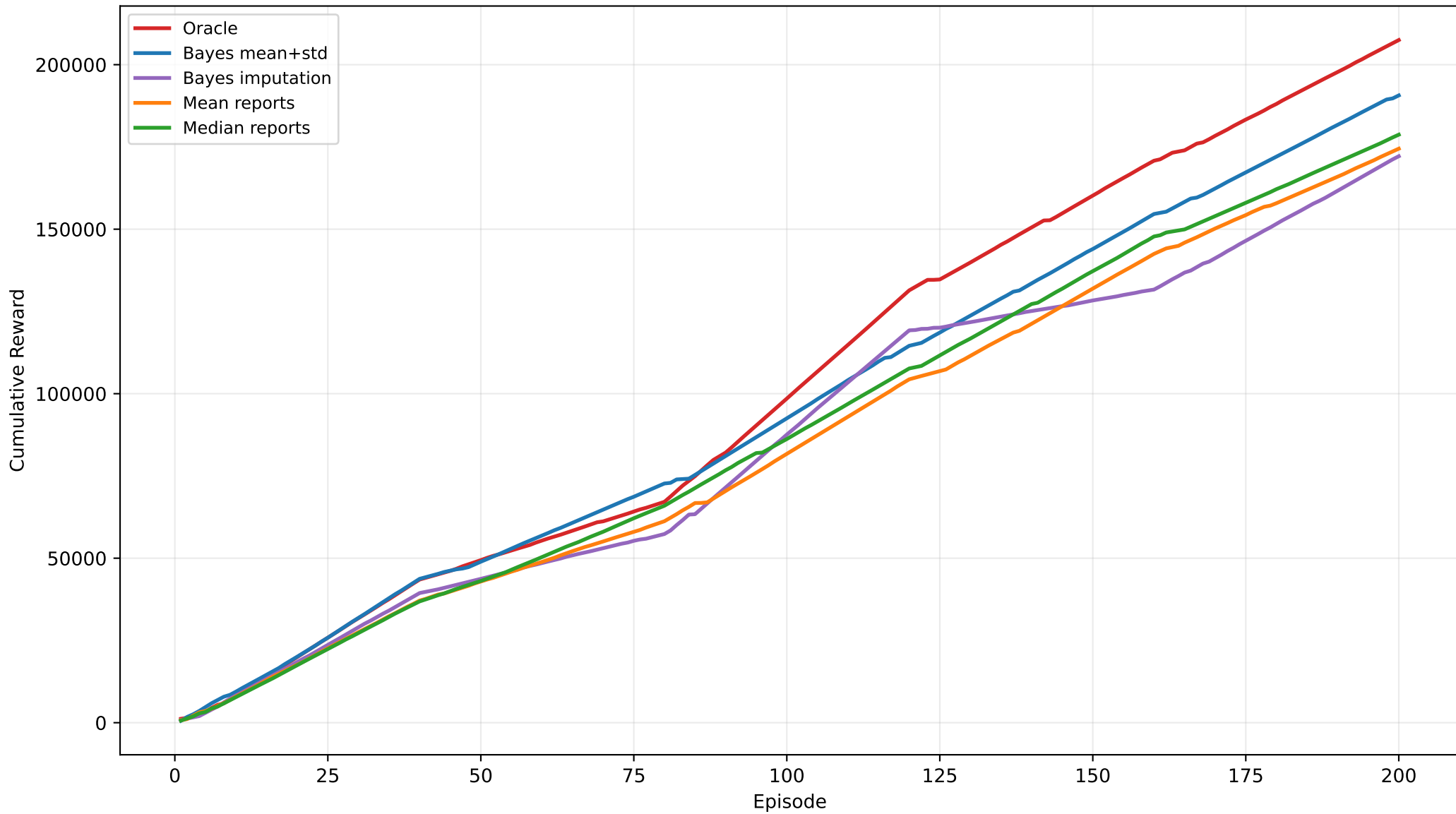
Mixed Unreliable: Cumulative Regret



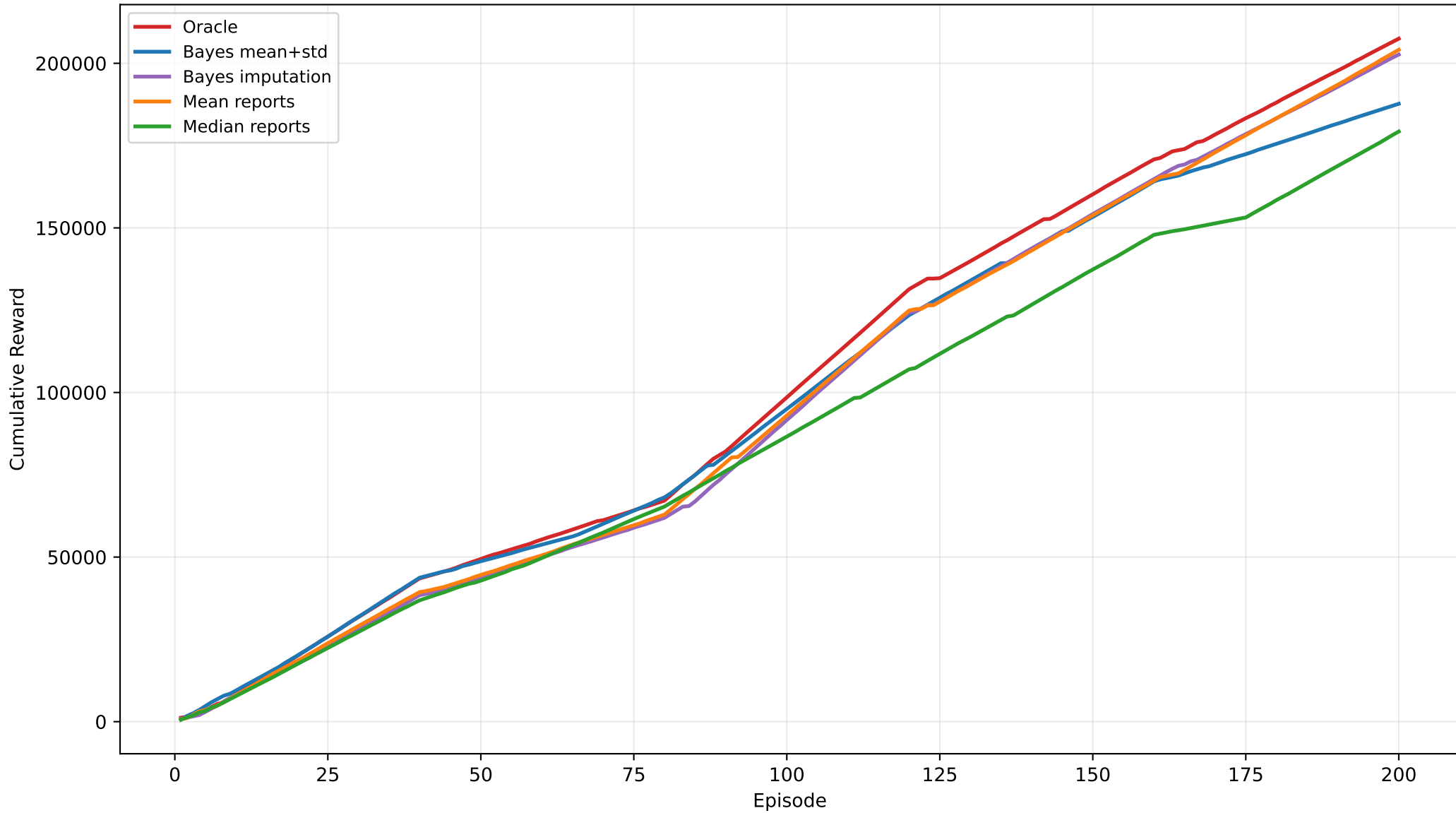
Biased Outliers: Cumulative Reward



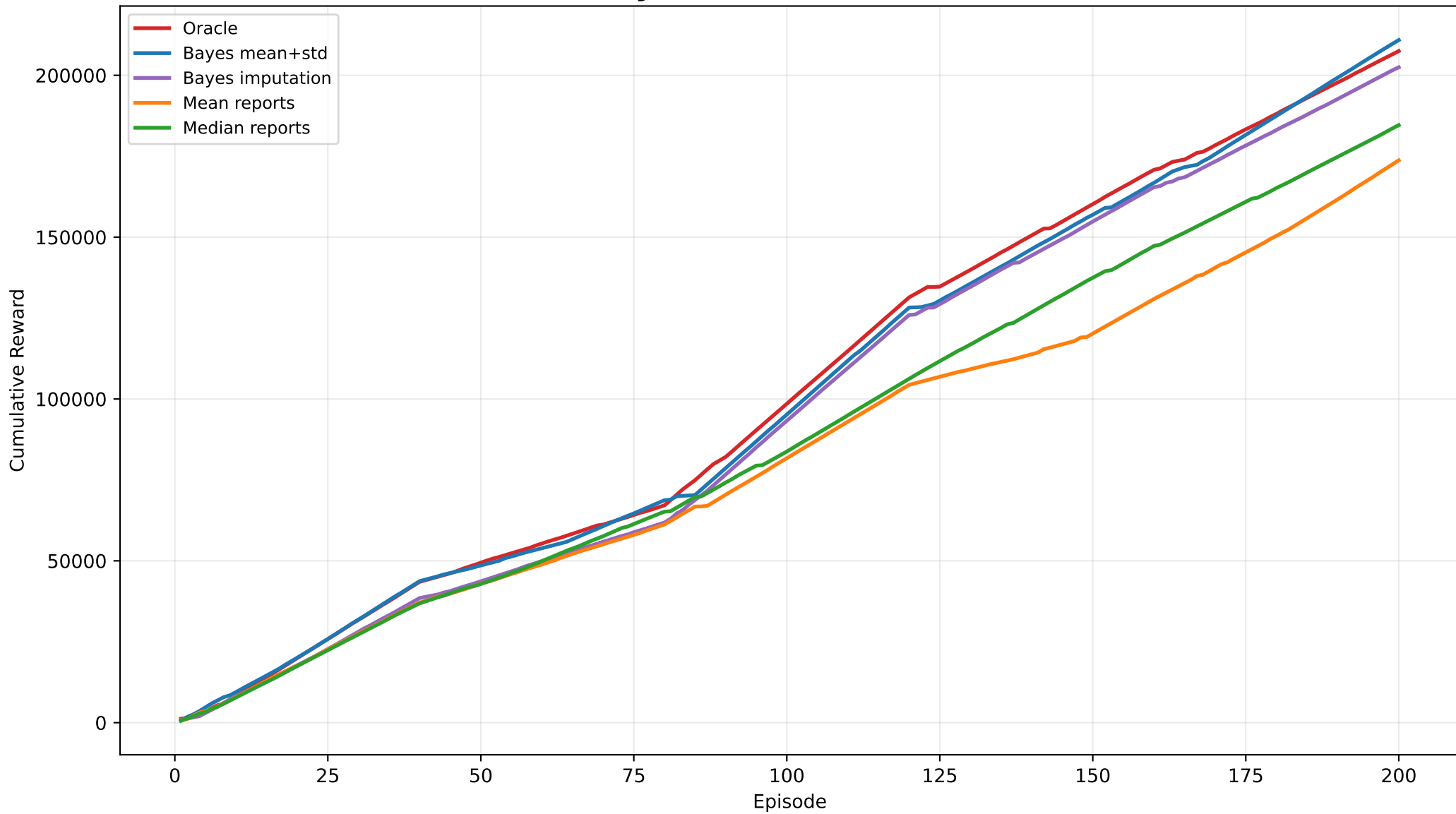
Persistent Bias: Cumulative Reward



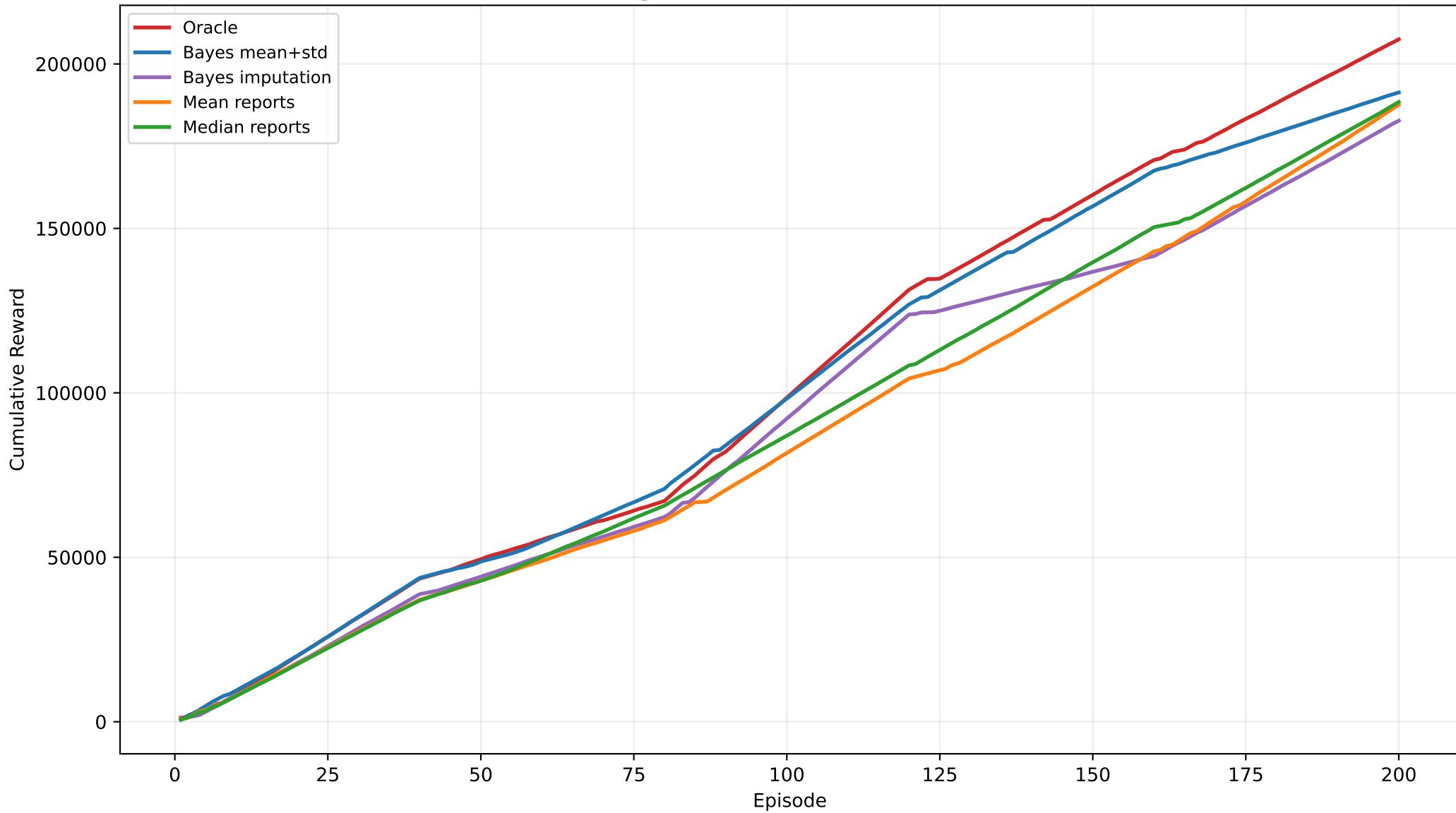
Heterogeneous Noise: Cumulative Reward



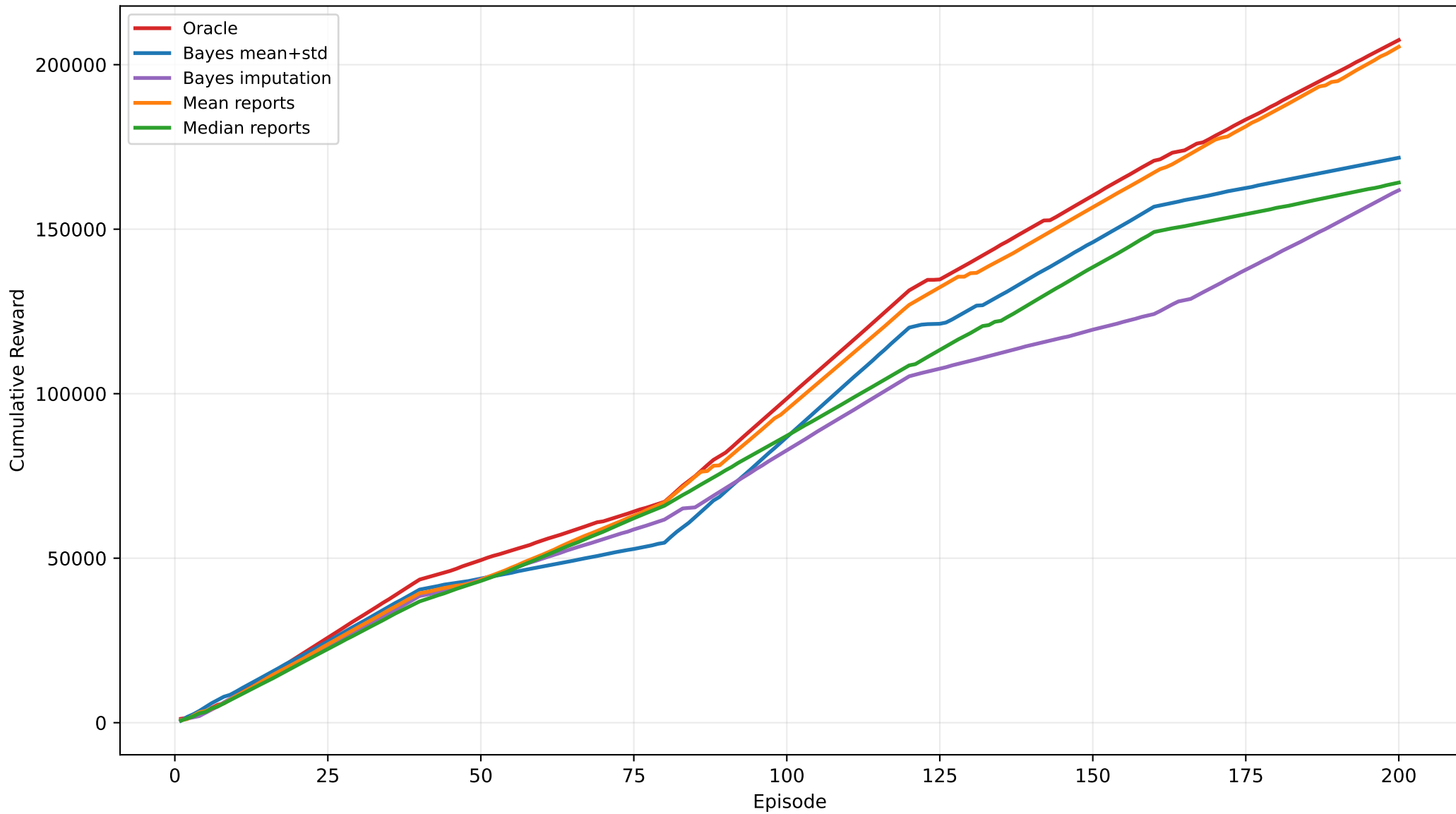
Reliability Drift: Cumulative Reward



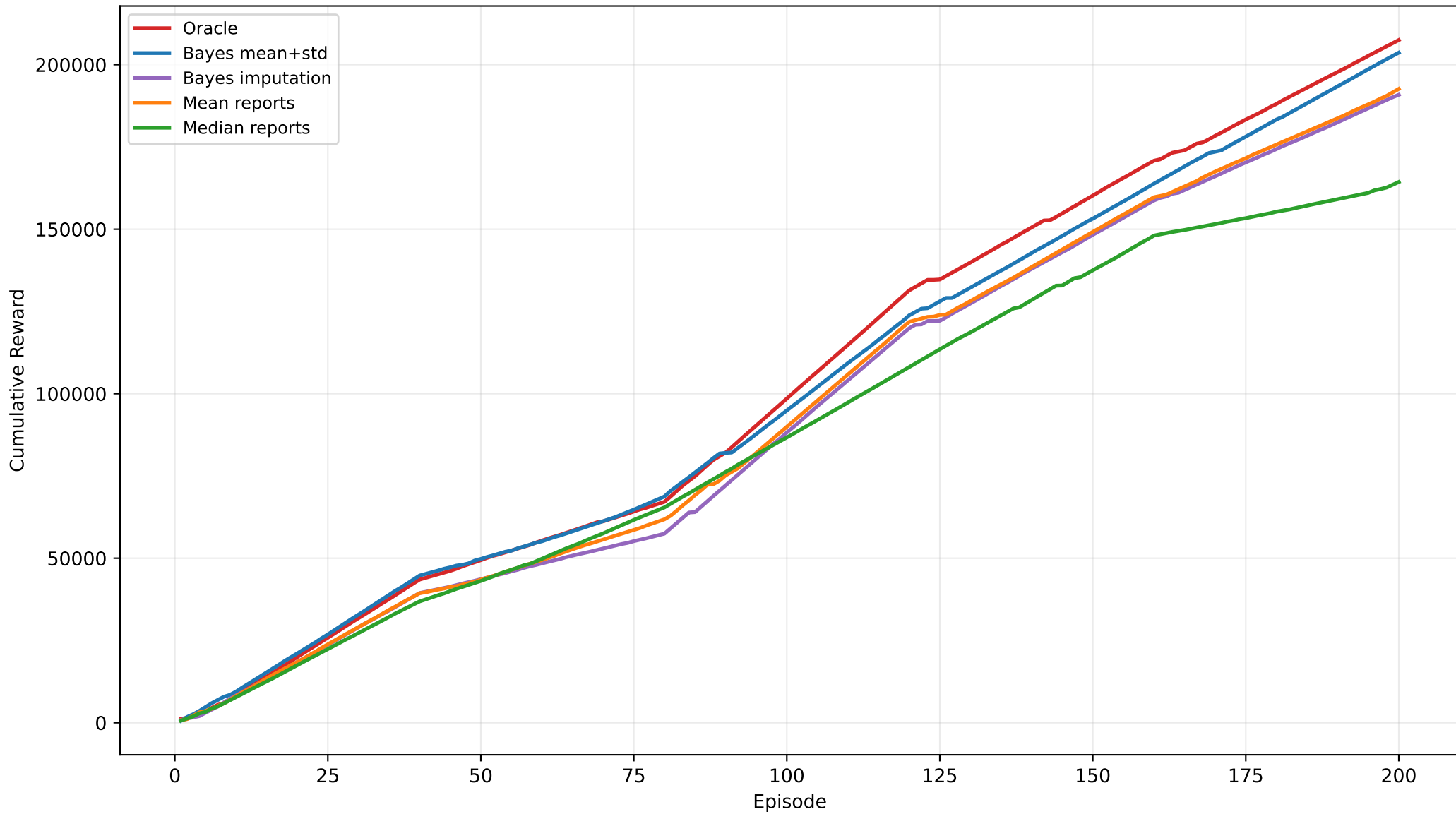
Stale Reports: Cumulative Reward



Outlier Bursts: Cumulative Reward



Mixed Unreliable: Cumulative Reward



Takeaways

The uncertainty-feature model is the strongest Bayesian variant in the current implementation. It is especially compelling when node reliability changes over time, because posterior uncertainty carries useful information for the action-value model.

Multiple imputation is weaker here. It can inject label noise by assigning one observed reward to several sampled latent contexts. The posterior can be useful, but the way it is fed into the learner matters.

Simple baselines remain important. Mean and median aggregation are competitive in several scenarios, which makes the evaluation more credible. The proposed contribution should be framed as an uncertainty-aware latent-context AdaChain extension, with strongest evidence under reliability drift and structured unreliable reporting.